

FitBuddy AI

5. Project Development Phase

Implementation record and module-level development description.

Backend Implementation

- Create the FastAPI application entry point.
- Create routes for the input form and plan generation.
- Define validation models where appropriate.
- Connect route logic to the Gemini service layer.
- Return generated data to Jinja2 templates.

Gemini Integration

- Load the API key from an environment variable.
- Initialize the Gemini client using the project's chosen SDK.
- Construct prompts from validated user data.
- Request a workout and nutrition plan.
- Handle API/network/model errors without exposing credentials.

Prompt Design

- State the role of the AI clearly.
- Provide the user's structured fitness information.
- Define the required sections of the response.
- Ask for practical, readable and internally consistent output.
- Avoid requesting unsafe or medically definitive recommendations.

Frontend Implementation

- Build the main input form with clear labels.
- Use Jinja2 to render dynamic values.
- Create a result page with separate workout and nutrition sections.
- Add responsive CSS and optional client-side validation.

Configuration

- Keep secrets in .env or another local secret store.
- Commit a .env.example containing placeholder variable names only.
- Keep dependencies in requirements.txt.
- Exclude virtual environments, caches and secrets from Git.

Development Deliverables

- Working source code.
- Templates and static assets.
- Configuration examples.

- Dependency file.
- Basic automated/manual test evidence.